

ER Diagram

25 September 2026 08:35

An ER Diagram(Entity-Relationship Diagram) is a graphical representation of the data, entities, attributes, and relationships in a database.

It is mainly used during the database design phase to understand how data will be organized before creating tables in a DBMS such as MySQL, Oracle, PostgreSQL, or SQL Server etc.

Why do we need an ER Diagram?

Suppose we want to create a database for a college management System.

The college needs to store information about:

- Students
- Teachers
- Courses
- Departments
- Subjects
- Enrollments

Before creating database tables, we need to understand:

- What information should be stored?
- What are the important objects?
- What properties does each object have?
- How the objects related?
- How many objects can participate in a relationship?

An ER diagram help us answer these questions visually.

For example:

STUDENT -----Enrolls-----COURSE

This tells us that students enroll in course.

Basic Components of an ER Diagram

The major components of an ER diagram are:

1. Entity
2. Attribute
3. Relationship
4. Key Attribute
5. Cardinality
6. Participation

1. Entity: An entity is a real world object or concept about which we want to store information.

Student
Teacher
Employee
Department

Course
Customer
Product
Bank Account

For example, in a college Database:

"Student" is an entity, because we want to store information about students.

Entity Representation

In a traditional ER diagram, an entity is represented by a rectangle.

Student

Entity Set: An Entity set is a collection of similar entities.

For example:

"Student" is an entity type/set, while individual students are entities.

Suppose a college has:

101	Rahul
102	James
103	Joy

Each individual student is an entity, while all students together form the Student entity set.

2. Attribute: An attribute describes a property or characteristics of an entity.

For example, a student can have:

Student

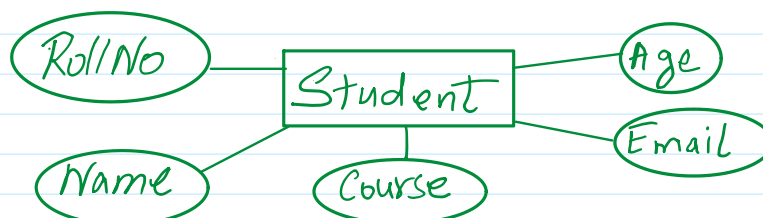
|
+--RollNo
+--Name
+--Age
+--Email
+--Course

Here:

- ☐ RollNo is an Attribute
- ☐ Name is an attribute.
- ☐ Age is an attribute.
- ☐ Email is an attribute.
- ☐ Course is an attribute.

Representation

An attribute is traditionally represented by an Oval/ellipse.



Types of Attributes

There are several important types of attributes.

Simple Attribute

A simple attribute cannot be meaningfully divided into smaller attributes.

Example:

Age
Gender
Salary
RollNo

Composite Attribute

A composite attribute. Can be divided into smaller attributes.

For example:

"Name"

Can be divided into:

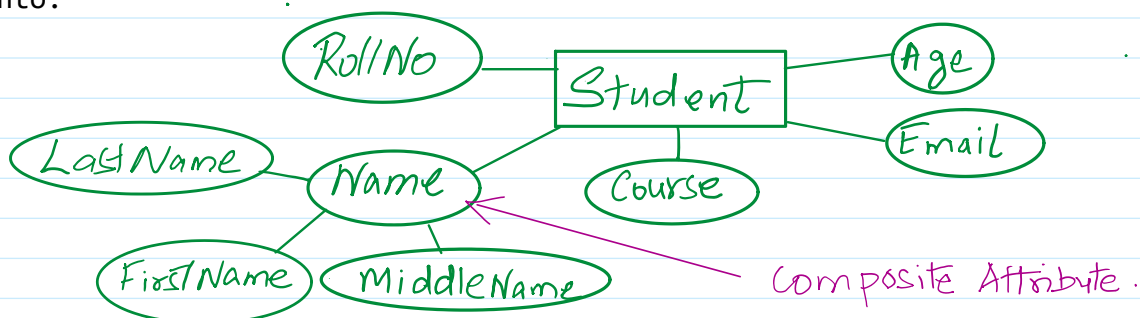
FirstName
MiddleName
LastName

Similarly:

"Address"

Could contain:

HouseNo
Street
City
State
Pincode



Single-Valued Attribute

A single-valued attribute has only one value for a particular entity.

Example:

Student -> DateOfBirth

A student normally has one date of birth.

Example:

RollNo
DateOfBirth
GreenCardNumber
EmployeeID
VisaID

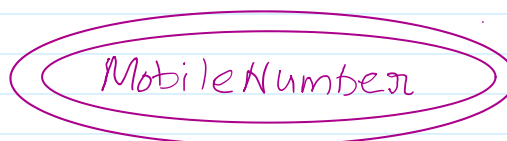
Multivalued Attribute

A multi valued attribute can have multiple values for one entity.

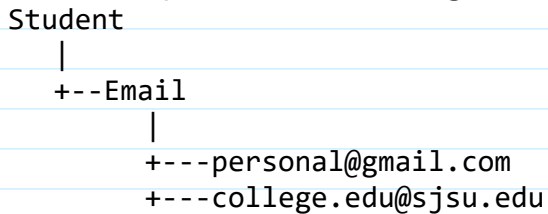
For example, an employee may have multiple mobile numbers:

Employee

|
+-- MobileNumber
|
+-- 87654578765
+-- 23354678566



In an ER diagram a multi valued attribute is represented using a double Oval. Another example, a student might have multiple emails:



Depending on the database requirements.

Derived Attributes

A derived attribute is an attribute whose values can be calculated from another attribute.

Example: DateOfBirth -> Age

If the date of birth is known, age can be calculated.

In an ER diagram, a derived attribute is represented using a dashed Oval.

Age

Another example:

(Salary + Bonus) -> totalSalary

totalSalary could be derived.

Key Attributes: A key attribute uniquely identifies an entity.

For example, suppose we have:

Student

RollNo

Name

Age

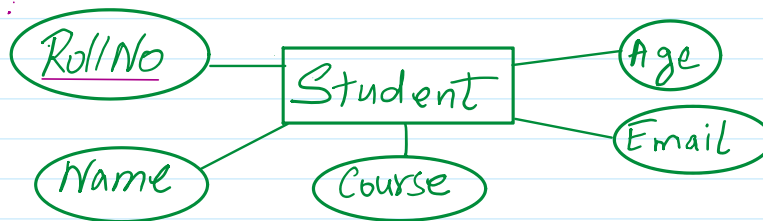
Email

Two students could have the same name, but their **RollNo** should uniquely identify them.

Therefore, **RollNo** can be the primary key.

ER representation

A key attribute is traditionally represented by an underline attribute.



Here RollNo is underlined, hence it is a Primary Key.

Relationship

A relationship represents an association between two or more entities.

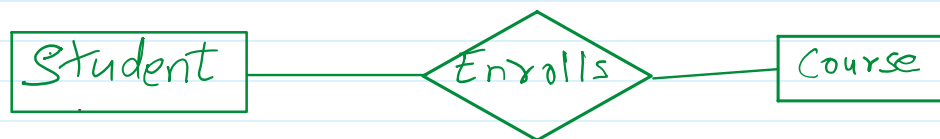
For example:

Student ---- Enrolls-----Course

Here:

- Student is an entity
- Course is an entity
- Enrolls is a relationship

In traditional ER diagram, a relationship is represented by a diamond.



ER relationship diagram.

Relationship Name

A relationship should normally have a meaningful name.

Examples:

Entity1	Relationship	Entity2
Student	Enrolls	Course
Employee	Works_For	Department
Customer	Places	Order
Doctor	Treats	Patient
Teacher	Teaches	Subject
Customer	Purchases	Product

Degree of Relationship

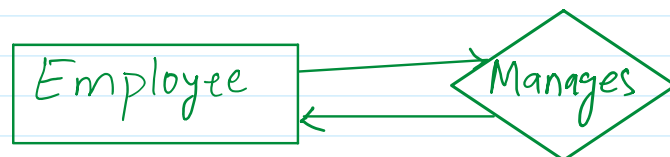
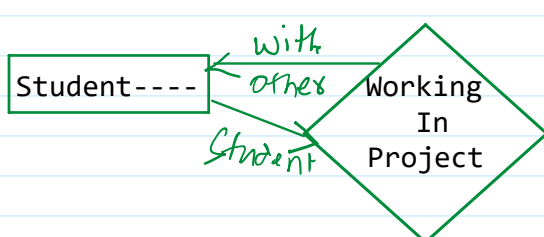
The degree of a relationship specifies how many entity types participate in the relationship.

The major types are:

1. Unary
2. Binary
3. Ternary

Unary Relationship

A unary relationship involves one entity type.



The same entity participates in the relationship in different roles:
Manager

Employee